

Hand-Over Document

Jugo IBCS Compliance AI Tool

Project: Jugo AI Tool — IBCS Scenario Unification Checker

Researchers: Ali, Ayoub, Dimitar, Iris, Nahia and Ruben

Stakeholders: Mohamed Abouyakob, Christian van Bracht

Date: 25/06/2026

Table of contents

What This Project Is About.....	3
What Was Built.....	3
Model	4
Dataset.....	4
Results	5
Feedback Generation	5
Saved Model.....	6
What Was Not Implemented.....	6
Known Limitations.....	6
Recommended Next Steps	7
Technology Stack.....	7

What This Project Is About

Organizations use dashboards and charts to analyze business data, but dashboards are often designed inconsistently, which makes them harder to interpret. IBCS (International Business Communication Standards) is a framework that addresses this by providing rules to standardize how business data is visualized, so that anyone reading a dashboard can interpret it the same way regardless of the company or tool used.

One of these rules is **UN 3.2 — Unify Scenarios**, which defines how different scenario types should be visually represented in charts. Scenarios like Actual, Previous Year, Plan, Budget and Forecast each have their own specific fill style, pattern and abbreviation so they are always visually distinguishable.

The goal of this project was to build an AI-powered tool for Jugo that can analyze a dashboard image and check whether it follows this rule. Because IBCS covers a lot of ground, the scope was kept to this one rule as a proof of concept.

What Was Built

A supervised multi-label image classification model that takes a dashboard image as input and checks it against ten predefined IBCS scenario rules. For each rule it outputs whether a mistake was detected (1) or not (0).

The ten labels are:

Label	What it checks
-------	----------------

AC-graph	Actual scenario visual notation
----------	---------------------------------

PY-graph	Previous Year visual notation
----------	-------------------------------

PL-graph	Plan visual notation
----------	----------------------

FC-graph	Forecast visual notation
----------	--------------------------

AC-abr	Abbreviation "AC" present and correct
--------	---------------------------------------

PY-abr	Abbreviation "PY" present and correct
--------	---------------------------------------

BU-abr	Abbreviation "BU" present and correct
--------	---------------------------------------

PL-abr	Abbreviation "PL" present and correct
--------	---------------------------------------

FC-abr	Abbreviation "FC" present and correct
--------	---------------------------------------

Axis	Semantic scenario notation on chart elements or variance axes
------	---

When a label is 1, it means a mistake was detected for that rule. When it is 0, it means either the rule is followed, not applicable, or the scenario is not present in the dashboard.

Model

Architecture: Pretrained ResNet18 (transfer learning), fine-tuned on the IBCS dataset. The original output layer was replaced with:

- A dropout layer (rate: 0.3)
- A linear layer with 10 outputs

Loss function: BCEWithLogitsLoss (suitable for multi-label classification)

Optimiser: Adam, learning rate 0.0001

Threshold: 0.5 per label

Batch size: 16

Max epochs: 30 with early stopping after 5 epochs without improvement on validation loss

Image preprocessing:

- Loaded with Pillow, converted to RGB
- Resized to 224×224 pixels
- Converted to a PyTorch tensor

Note: no data augmentation was applied. ImageNet normalisation was also not applied, even though ResNet18 weights were pretrained on ImageNet.

Dataset

- Images are loaded from: ../images_data/ibcs-images
- Labels are read from: ../images_data/trial-correct.csv
- Split: 70% training / 15% validation / 15% test
- **No random seed was set**, so re-running the notebook will produce a different split and potentially different results

The dataset was created manually. No publicly available IBCS-labeled dataset existed, so images were gathered through web scraping and an initial set of 27 compliant dashboards provided by the stakeholders.

Results

Metric	Score
Training accuracy	~99–100%
Validation accuracy	~90%
Test accuracy	~90%
Micro F1	0.71
Macro F1	0.19
Weighted F1	0.57
Sample F1	0.52

The overall accuracy is misleading. Because most labels are 0, a model can score high by predicting 0 for everything. The model currently performs well mainly on AC-graph (F1: 1.00) and AC-abr (F1: 0.94). For the remaining labels, F1-scores were 0 in the recorded test run, partly because some labels had very few or zero positive examples in the test split.

Feedback Generation

The model itself does not generate written text. A Python dictionary maps each of the ten outputs to a fixed feedback message. For example, when AC-graph is flagged, the system returns:

"The dashboard does not correctly represent Actual values using dark solid notation."

If no label exceeds the 0.5 threshold, the system reports that no anomaly was detected and the dashboard is likely compliant. This does not guarantee compliance — it only means none of the learned anomalies were detected above the threshold.

Saved Model

The model is saved as `ibcs_final_model.pth`. Only the PyTorch `state_dict` is stored — this means only the weights are saved, not the architecture, labels, or preprocessing steps.

To load the model correctly, a future developer must:

1. Recreate a ResNet18 model
2. Replace the final layer with dropout 0.3 and a linear layer with 10 outputs
3. Load the saved state dictionary
4. Apply the same image transformation (resize to 224×224, convert to tensor, no normalisation)
5. Keep the exact label order as listed above
6. Apply sigmoid and the 0.5 threshold
7. Connect predictions to the feedback dictionary

What Was Not Implemented

Object detection was not used. The model classifies the dashboard image as a whole and does not locate where in the image a mistake occurs. This was a deliberate decision due to time constraints. Despite this, the model already shows reasonable performance for the rules it was able to learn. A future developer could extend the system with object detection (e.g. YOLO or Faster R-CNN) to also indicate the location of mistakes within a dashboard.

Known Limitations

- The dataset is small and imbalanced across the ten labels
- Some labels have very few or no positive examples in the test set
- The train/test split is not reproducible (no random seed) and not stratified
- No data augmentation was implemented
- ImageNet normalisation was not applied
- One threshold (0.5) is used for all labels regardless of class frequency
- The model only detects the ten predefined anomalies — it cannot detect unknown compliance issues
- A missing Plan or Forecast scenario is hard to detect from an image alone

Recommended Next Steps

- Collect and label more dashboard images, especially for PL, FC, PY and Axis rules
- Set a random seed for reproducibility
- Implement a stratified multi-label split
- Apply ImageNet normalisation and data augmentation
- Test per-label thresholds instead of a single global threshold
- Evaluate using precision, recall and F1-score per rule rather than overall accuracy
- Test on an external dataset with dashboard styles not seen during training
- Explore object detection to localize mistakes within dashboard images

Beyond the points above, it is also worth testing class weighting to handle the label imbalance. Once the model is more mature, evaluating it on a fully external test set with dashboard styles and sources never seen during training would give a much more realistic picture of how it performs in practice.

Technology Stack

Python, Jupyter Notebook, PyTorch, Torchvision, ResNet18, Pillow, Pandas, NumPy, Scikit-learn, Matplotlib, Git/GitHub, CUDA (when available)